



Muhammad Nawaz Sharif University of Engineering and Technology, Multan

LAB MANUAL Artificial Intelligence

Name

M.JUNAID

Reg. No

2023-IT-39

Session

2023-2027

Class

BS-IT (4th semester)

Section

Evening-B

Submitted to

MAM NASHITAH ALWAZ

LAB NO#01

What is Python?

Python is a high-level, interpreted and general-purpose programming language. Here's what that means:

- High-level _Easy to read and write, closer to human language.
- Interpreted _Code runs line by line instead of being compiled all at once.
- General-purpose _Can be used for web development, data science, machine learning, automation and more.

Why Learn Python?

- Simple & Easy -Its syntax (rules) is straightforward, making it easy to understand.
- Widely Used -Big companies like Google, Facebook, and Microsoft use Python.
- Powerful Libraries -Ready-made tools (like NumPy , Pandas, and Tensor Flow) make complex tasks easier.
- Great for Beginners -You can start coding with just a few lines!

Initiation of coding on python

Step 1: Install Python

Before we start coding, we need to install Python on your computer.

For Windows Users:

- Download Python -Go to (python.org/downloads) and click the big yellow download button (it will automatically choose the right version for your system).
- Install Python -Open the downloaded file and check the box that says "Add Python to PATH" (this is important!).Then click Install Now and wait for it to finish.
- Check if Python is installed -Open Command Prompt (CMD) and type:
- `python--version`
- If you see something like Python 3.12.0, congrats!You're good to go.

For Mac Users:

- Open Terminal(press Cmd + Space, type Terminal and hit Enter).
- Type brew install python
- (If you don't have Homebrew, first install it from brew.sh).
- Check if Python is installed with: python3--version

For Linux Users (Ubuntu/Debian):

- Open Terminal and type: sudo apt update
- sudo apt install python3
- Check if it's installed: python3--version

Step 2: Run Python on Your PC

Once Python is installed, you have two ways to run it:

1. Quick Test in Python's Built-in Console (REPL)

- Open Command Prompt (Windows)/Terminal (Mac/Linux)
- Type python (or python3 on Mac/Linux) and press Enter
- You'll see something like this:
- That's Python's interactive mode! Now type:

```
print("Hello, Python!")
```

and press Enter.

If it prints:

Hello, Python!

Then Python is working.

2. Running a Python Script (Recommended)

- Open Notepad (Windows) or Text Edit (Mac)
- Type this simple program:

```
print("Welcome to Python!")
```
- Save it as hello.py (make sure the extension is .py).
- Open Command Prompt (Windows)/Terminal (Mac)
- Navigate to the file's location using cd (change directory) command. Example:
cd Desktop

- Run the program by typing:
`python hello.py`
- If it prints:
Welcome to Python!
- You did it!

Step 3: Install a Better Code Editor (Optional but Useful)

Writing Python in Notepad is possible but not the best experience. Here are some better tools:

- VS Code (Best for beginners & pros) -Download
- PyCharm (Great for Python development) -Download
- Jupyter Notebook (For data science & ML) -Install via `pip install jupyter`
- Once installed, open any of these, type your Python code, and press Run.
- You're Ready to Code!

Understanding Variables and Data Types in Python

Objective:

- To learn how to declare and use variables.
- To understand different data types in Python.

Theory:

In Python, a variable is a container for storing data values. You don't need to declare the data type explicitly; Python automatically identifies it.

Data Types are the classification of data items. Some basic data types are:

- `int` (integer numbers)
- `float` (decimal numbers)
- `str` (string/text)
- `bool` (True/False values)
- `list`, `tuple`, `dict` (collection data types)

```

# Integer type
age = 20
print("Age:", age)
print("Type of age:", type(age))

# Float type
height = 5.6
print("Height:", height)
print("Type of height:", type(height))

# String type
name = "Junaid"
print("Name:", name)
print("Type of name:", type(name))

# Boolean type
is_student = True
print("Is Student:", is_student)
print("Type of is_student:", type(is_student))

# List type
subjects = ["Math", "Physics", "Chemistry"]
print("Subjects:", subjects)
print("Type of subjects:", type(subjects))

# Tuple type
marks = (85, 90, 95)
print("Marks:", marks)
print("Type of marks:", type(marks))

# Dictionary type
student = {"name": "Junaid", "age": 20, "course": "BS IT"}
print("Student Info:", student)
print("Type of student:", type(student))

```

```

→ Age: 20
Type of age: <class 'int'>
Height: 5.6
Type of height: <class 'float'>
Name: Junaid
Type of name: <class 'str'>
Is Student: True
Type of is_student: <class 'bool'>
Subjects: ['Math', 'Physics', 'Chemistry']
Type of subjects: <class 'list'>
Marks: (85, 90, 95)
Type of marks: <class 'tuple'>
Student Info: {'name': 'Junaid', 'age': 20, 'course': 'BS IT'}
Type of student: <class 'dict'>

```

Basic concepts:

- **int (Integer):** Whole numbers, e.g., -3, 0, 42
- **float (Floating-point number):** Decimal numbers, e.g., 3.14, 0.99, -0.05
- **str (String):** Sequence of characters (text), e.g., "Hello", "123"
- **bool (Boolean):** Represents True or False values, e.g., True, False
- **list:** Ordered, mutable collection of items, e.g., [1, 2, 3], ["apple", "banana"]
- **tuple:** Ordered, immutable collection of items, e.g., (1, 2, 3), ("a", "b", "c")
- **dict (Dictionary):** Collection of key-value pairs, e.g., {"name": "Junaid", "age": 20}
- **set:** Unordered collection of unique elements, e.g., {1, 2, 3}, {"apple", "banana"}

Understanding Conditional Statements in Python

Objective:

- To learn how to use if, else, and elif statements for decision-making in Python.
- To understand how Python evaluates conditions and executes code based on those conditions.

Theory:

Conditional statements in Python allow you to execute different blocks of code based on specific conditions. The most common conditional statements are:

- **if:** Executes a block of code if a condition is True.
- **else:** Executes a block of code if the if condition is False.
- **elif (else if):** Used to check multiple conditions when the if condition is False.

if Statement:

The if statement is used to check a condition and execute a block of code only if the condition is True.

Explanation:

- The if statement evaluates an expression (condition). If the condition evaluates to True, the block of code inside the if statement runs.
- If the condition is False, the program skips the if block and moves to the next block (like elif or else).

Example:

```
if number > 0:
```

```
    print("The number is positive.")
```

- In this example, if number is greater than 0, it prints the message that the number is positive.

elif Statement:

The elif statement stands for "else if" and is used when we need to check multiple conditions. It follows the if statement and is executed only if the if condition is False.

Explanation:

- You can have multiple elif conditions after an if statement.
- When the first if condition fails, Python checks the elif condition(s) in order.
- Once a condition is found to be True, its corresponding block of code is executed. If none of the elif conditions are True, the program will execute the else block (if provided).

Example:

```
if number > 0:
```

```
    print("The number is positive.")
```

elif number < 0:

```
print("The number is negative.")
```

- If number is greater than 0, it prints that the number is positive.
- If number is less than 0, it prints that the number is negative.

else Statement:

The else statement is the final block in a conditional statement, executed when all preceding if or elif conditions are False.

Explanation:

- The else block is optional, but when used, it ensures that a block of code runs when none of the conditions are True.
- It's the default action when all other conditions are skipped.

Example:

if number > 0:

```
print("The number is positive.")
```

elif number < 0:

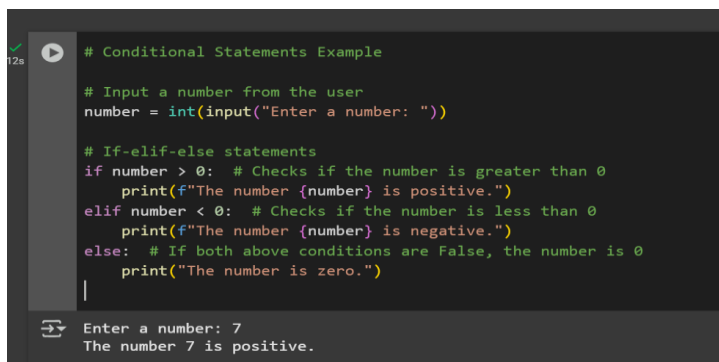
```
print("The number is negative.")
```

else:

```
print("The number is zero.")
```

- In this case, if neither the if nor elif conditions are True, the else block will execute, meaning the number is 0.

Code Screen shot to verify conditional statements:



```
# Conditional Statements Example

# Input a number from the user
number = int(input("Enter a number: "))

# If-elif-else statements
if number > 0: # Checks if the number is greater than 0
    print(f"The number {number} is positive.")
elif number < 0: # Checks if the number is less than 0
    print(f"The number {number} is negative.")
else: # If both above conditions are False, the number is 0
    print("The number is zero.")

Enter a number: 7
The number 7 is positive.
```

Understanding Loops in Python (for loop and while loop)

Objective:

- To learn how to repeat actions using loops in Python.
- To understand the working of for loops and while loops.

Theory:

Loops allow us to repeat a block of code multiple times without writing it again and again.

There are two main types of loops in Python:

For Loop:

A for loop is used to iterate (repeat) over a sequence (like a list, tuple, dictionary, string, or range).

Explanation:

- The for loop automatically picks each item from a collection and runs the code block once for each item.
- It's mainly used when you know beforehand how many times you want to repeat something.

Syntax:

for variable in sequence:

 # block of code

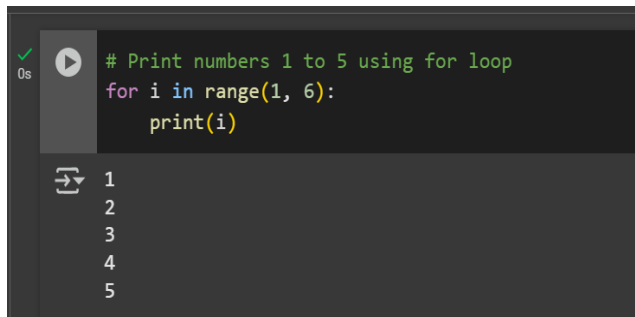
Example:

```
for i in range(5):
```

```
    print(i)
```

- This will print numbers 0 to 4.

Screen shot:



```
0s ✓ # Print numbers 1 to 5 using for loop
for i in range(1, 6):
    print(i)
```

1
2
3
4
5

While Loop:

A while loop keeps executing a block of code as long as the given condition is True.

Explanation:

- The while loop is mainly used when you don't know beforehand how many times you need to repeat the block — it depends on a condition.
- The loop stops when the condition becomes False.

Syntax:

while condition:

 # block of code

Example:

count = 0

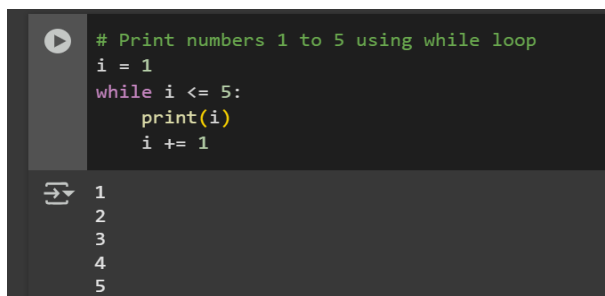
while count < 5:

 print(count)

count += 1

This will also print numbers 0 to 4.

Screen shot:



```
# Print numbers 1 to 5 using while loop
i = 1
while i <= 5:
    print(i)
    i += 1
```

1
2
3
4
5

Understanding Functions in Python

Objective:

- To learn how to define and use functions in Python.
- To understand the importance of reusing code using functions.

Theory:

Functions are blocks of organized, reusable code that perform a specific task.

Instead of writing the same code again and again, we can define a function once and call it whenever needed.

Definition:

A function is a named block of code that runs only when it is called.

Explanation:

- Functions make programs more organized and manageable.
- They allow code reuse, making programs shorter and easier to understand.

Python has two types of functions:

- **Built-in Functions:** Already available in Python (like `print()`, `len()`, `input()`, etc.).
- **User-defined Functions:** Functions created by the programmer.

How to Define a Function:

Syntax:

```
def function_name(parameters):
```

```
    # block of code
```

```
    return result
```

- `def` keyword is used to define a function.
- `function_name` is the name you give to the function.
- `parameters` are optional — they are inputs to the function.
- `return` is used to send back the output (optional).

Calling a Function:

After defining a function, you must call it to execute the code inside it.

Example:

```
def greet():
```

```
    print("Hello, welcome to Python!")
```

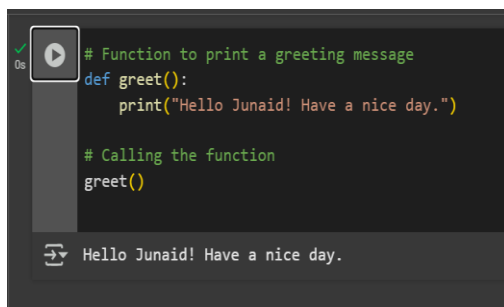
```
greet() # Function call
```

Algorithm/Steps:

- Use def keyword to define a function.
- Write the function name and parentheses ().
- Inside parentheses, optionally add parameters (inputs).
- Write the block of code inside the function.
- Call the function whenever needed.

Screenshots:

• Function without Parameters:



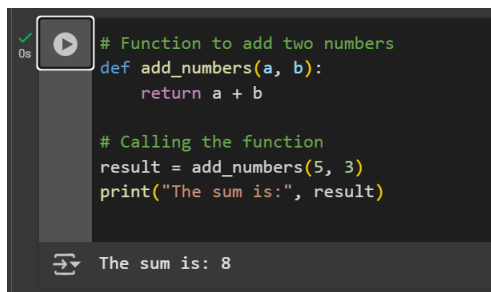
```
# Function to print a greeting message
def greet():
    print("Hello Junaid! Have a nice day.")

# Calling the function
greet()
```

0s [play icon]

⇒ Hello Junaid! Have a nice day.

• Functions with parameters:



```
# Function to add two numbers
def add_numbers(a, b):
    return a + b

# Calling the function
result = add_numbers(5, 3)
print("The sum is:", result)
```

0s [play icon]

⇒ The sum is: 8

Understanding Lists, Tuples, and Dictionaries in Python

Objective:

- To learn about Python's main data structures: List, Tuple, and Dictionary.
- To understand how to store and manage collections of data.

Theory:

Python provides powerful ways to store multiple items in a single variable.

The most commonly used data structures are:

1. List:

A list is an ordered, mutable (changeable) collection of items.

Items can be of different data types (integers, strings, floats, etc.).

Explanation:

- Lists are written with square brackets [].
- Elements are separated by commas.
- You can modify, add, or remove elements after the list is created.

Syntax:

```
list_name = [item1, item2, item3]
```

Example:

```
Names = ["Junaid Khan", "Adnan Rajpoot", "Amir"]
```

Screen shot:



```
# Creating a List
Names = ["Junaid Khan", "Adnan Rajpoot", "Amir"]
print("List of Names:", Names)

# Accessing elements
print("First Name:", Names[0])

# Adding a new fruit
Names.append("Tahir")
print("Updated list of Names:", Names)
```

Output:

```
List of Names: ['Junaid Khan', 'Adnan Rajpoot', 'Amir']
First Name: Junaid Khan
Updated list of Names: ['Junaid Khan', 'Adnan Rajpoot', 'Amir', 'Tahir']
```

2. Tuple:

A tuple is an ordered, immutable (unchangeable) collection of items.

Explanation:

- Tuples are written with round brackets ().
- Once a tuple is created, you cannot modify its contents.
- Used when you want to protect data from being changed.

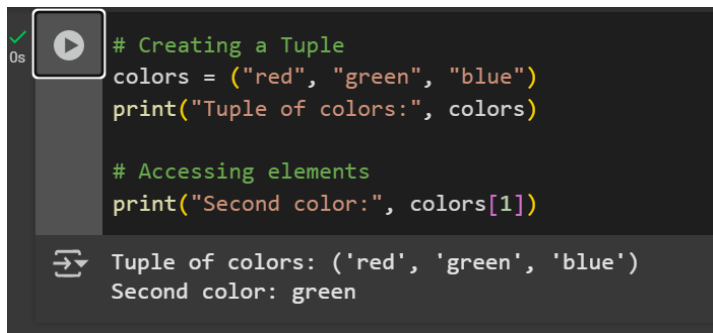
Syntax:

```
tuple_name = (item1, item2, item3)
```

Example:

```
colors = ("red", "green", "blue")
```

Screen shot:

A screenshot of a code editor with a dark background. The code is written in Python. It starts with a comment "# Creating a Tuple", followed by the line "colors = ('red', 'green', 'blue')", and then "print('Tuple of colors:', colors)". Below this is another comment "# Accessing elements", followed by "print('Second color:', colors[1])". The output of the code is shown at the bottom: "Tuple of colors: ('red', 'green', 'blue')" and "Second color: green".

```
# Creating a Tuple
colors = ("red", "green", "blue")
print("Tuple of colors:", colors)

# Accessing elements
print("Second color:", colors[1])
```

```
Tuple of colors: ('red', 'green', 'blue')
Second color: green
```

3. Dictionary:

A dictionary is an unordered collection of items.

Each item has a key and a value pair.

Explanation:

- Dictionaries are written with curly braces { }.
- Each key is unique and is used to access its corresponding value.
- Values can be of any type.

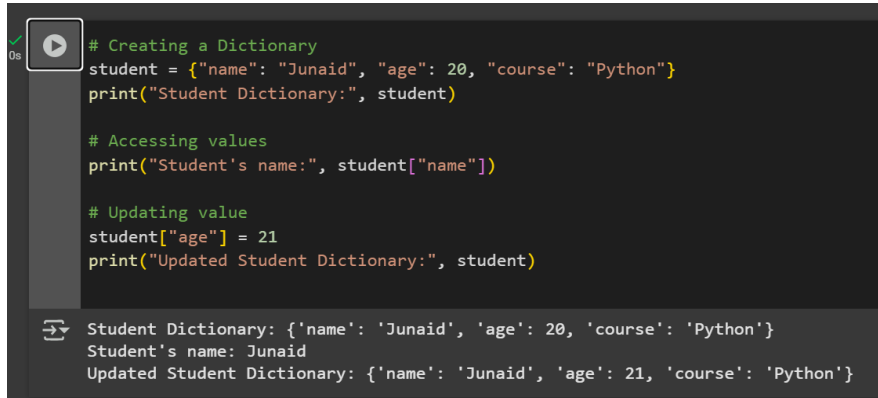
Syntax:

```
dict_name = {key1: value1, key2: value2}
```

Example:

```
student = {"name": "Junaid", "age": 20, "course": "Python"}
```

Screen shot:



```
# Creating a Dictionary
student = {"name": "Junaid", "age": 20, "course": "Python"}
print("Student Dictionary:", student)

# Accessing values
print("Student's name:", student["name"])

# Updating value
student["age"] = 21
print("Updated Student Dictionary:", student)
```

Student Dictionary: {'name': 'Junaid', 'age': 20, 'course': 'Python'}
Student's name: Junaid
Updated Student Dictionary: {'name': 'Junaid', 'age': 21, 'course': 'Python'}

LAB NO # 02

Introduction to NumPy Library in Python

Objective:

- To learn what NumPy is and why it is used.
- To understand basic NumPy operations like creating arrays.

Theory:

NumPy (Numerical Python) is a powerful library used for numerical computing in Python.

It provides support for:

- Working with large, multi-dimensional arrays and matrices.
- Performing mathematical operations faster and more efficiently.
- NumPy arrays are more powerful and faster than Python lists.

Definition:

NumPy is a Python library for working with arrays. It also has functions for working in linear algebra, statistics, and mathematics.

Explanation:

- Normal Python lists are slow for large data.
- NumPy arrays are faster and more memory efficient.
- Arrays can be 1-D, 2-D, or even multi-Dimensional.

Syntax to import NumPy:

```
import numpy as np
```

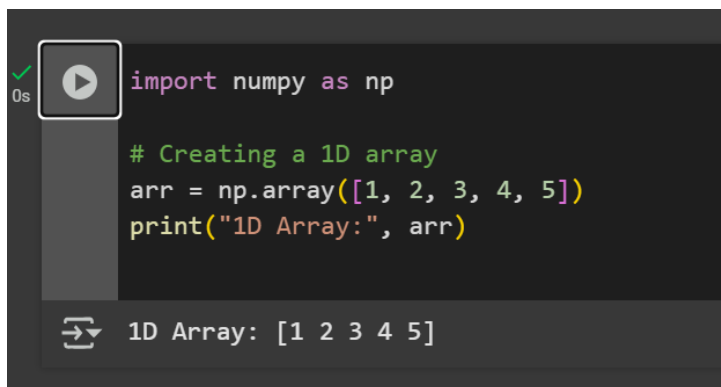
- np is the common short name (alias) for NumPy.

Algorithm/Steps:

- Install NumPy using pip install numpy (only if not already installed).
- Import NumPy in your code.
- Create a NumPy array.
- Perform basic operations like accessing elements.

Code Screen Shot:

1. Creating a 1D NumPy Array



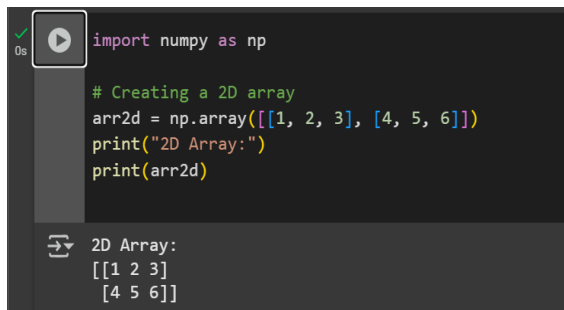
The screenshot shows a code editor with a dark background. On the left, there is a vertical toolbar with a play button icon. The code in the editor is as follows:

```
import numpy as np

# Creating a 1D array
arr = np.array([1, 2, 3, 4, 5])
print("1D Array:", arr)
```

Below the code, the output of the print statement is displayed: "1D Array: [1 2 3 4 5]".

2. Creating a 2D NumPy Array

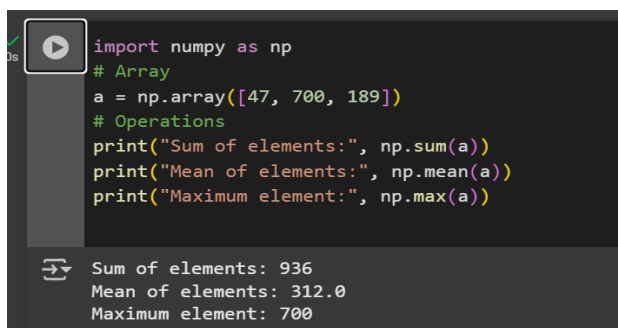


```
import numpy as np

# Creating a 2D array
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
print("2D Array:")
print(arr2d)
```

2D Array:
[[1 2 3]
 [4 5 6]]

3. Some Basic NumPy Operations



```
import numpy as np

# Array
a = np.array([47, 700, 189])

# Operations
print("Sum of elements:", np.sum(a))
print("Mean of elements:", np.mean(a))
print("Maximum element:", np.max(a))
```

Sum of elements: 936
Mean of elements: 312.0
Maximum element: 700

Introduction to Pandas Library in Python

Objective:

- To understand what Pandas is and how it is used for data handling.
- To learn basic Pandas operations like creating and displaying data.

Theory:

Pandas is an open-source Python library mainly used for data manipulation, analysis, and cleaning.

It allows easy handling of structured data like tables (like Excel sheets).

Pandas provides two main data structures:

- Series (1D data)
- DataFrame (2D data - rows and columns like a table)

Definition:

Pandas is a Python library that makes it easy to work with structured data.

Explanation:

- Useful for loading, analyzing, and manipulating big datasets easily.
- Allows reading from different file formats (CSV, Excel, SQL databases, etc.).
- Works well with missing data, filtering, and sorting.

Syntax to import Pandas:

```
import pandas as pd
```

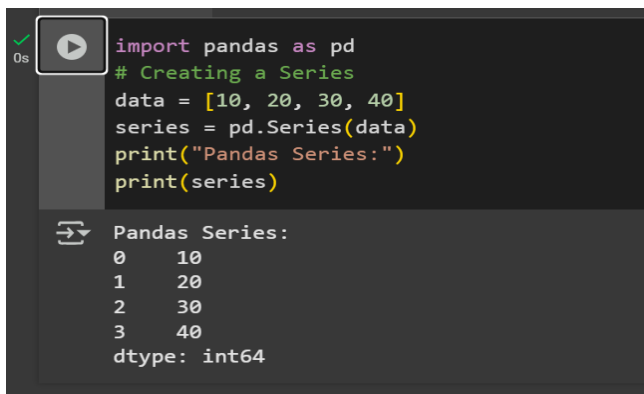
- pd is the short name (alias) used for Pandas.

Algorithm/Steps:

- Install Pandas using `pip install pandas` (if not already installed).
- Import Pandas in your code.
- Create a Series and a DataFrame.
- Perform basic operations like viewing data.

Code screenshot:

1. Creating a Pandas Series



The screenshot shows a Jupyter Notebook interface. The top part is a code editor with the following Python code:

```
import pandas as pd
# Creating a Series
data = [10, 20, 30, 40]
series = pd.Series(data)
print("Pandas Series:")
print(series)
```

The bottom part of the screenshot shows the output of the code, which is a Pandas Series:

```
Pandas Series:
0    10
1    20
2    30
3    40
dtype: int64
```

2. Creating a Pandas DataFrame

```
import pandas as pd
# Creating a DataFrame
data = {
    "Name": ["Jnaid", "Adnan", "Amir"],
    "Age": [20, 21, 38]
}
df = pd.DataFrame(data)
print("\nPandas DataFrame:")
print(df)
```

Pandas DataFrame:

	Name	Age
0	Jnaid	20
1	Adnan	21
2	Amir	38

3. Reading Data from a CSV file

```
import pandas as pd
# Reading data from CSV (if you have a file)
df = pd.read_csv('amazon_sales_data 2025.csv')
print(df)
```

	Order ID	Date	Product	Category	Price	Quantity	\
0	ORD0001	14-03-25	Running Shoes	Footwear	60	3	
1	ORD0002	20-03-25	Headphones	Electronics	100	4	
2	ORD0003	15-02-25	Running Shoes	Footwear	60	2	
3	ORD0004	19-02-25	Running Shoes	Footwear	60	3	
4	ORD0005	10-03-25	Smartwatch	Electronics	150	3	
...	...	...	...	...	...	...	
245	ORD0246	17-03-25	T-Shirt	Clothing	20	2	
246	ORD0247	30-03-25	Jeans	Clothing	40	1	
247	ORD0248	05-03-25	T-Shirt	Clothing	20	2	
248	ORD0249	08-03-25	Smartwatch	Electronics	150	3	
249	ORD0250	19-02-25	Smartphone	Electronics	500	4	
	Total Sales	Customer Name	Customer Location	Payment Method	Status		
0	180	Emma Clark	New York	Debit Card	Cancelled		
1	400	Emily Johnson	San Francisco	Debit Card	Pending		

Introduction to Matplotlib and Seaborn for Data Visualization

Objective:

- To learn how to create basic charts and graphs using Matplotlib and Seaborn libraries.
- To visualize data effectively for better understanding.

Theory:

Data Visualization means representing data in the form of charts, graphs, and plots.

It helps in:

- Understanding patterns and trends easily.

- Making data more readable and visually appealing.

Python has two powerful libraries for this:

Matplotlib: Basic plotting library (foundation).

Seaborn: Advanced visualization built on top of Matplotlib (more beautiful graphs).

Definition:

Matplotlib is a Python library used for creating static, animated, and interactive plots.

Seaborn is a Python library based on Matplotlib that makes beautiful and informative statistical graphics.

Explanation:

- Matplotlib is useful when you want full control over every element of a figure.
- Seaborn is simpler for complex visualizations like distribution plots, heatmaps, etc.

Syntax to import:

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

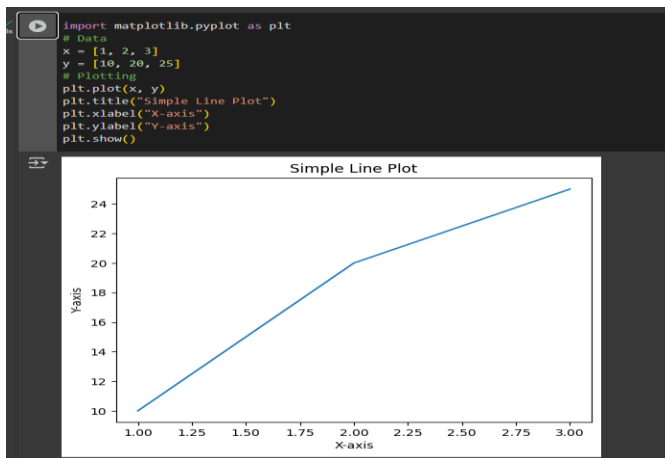
- plt is used for Matplotlib plotting.
- sns is used for Seaborn plotting.

Algorithm/Steps:

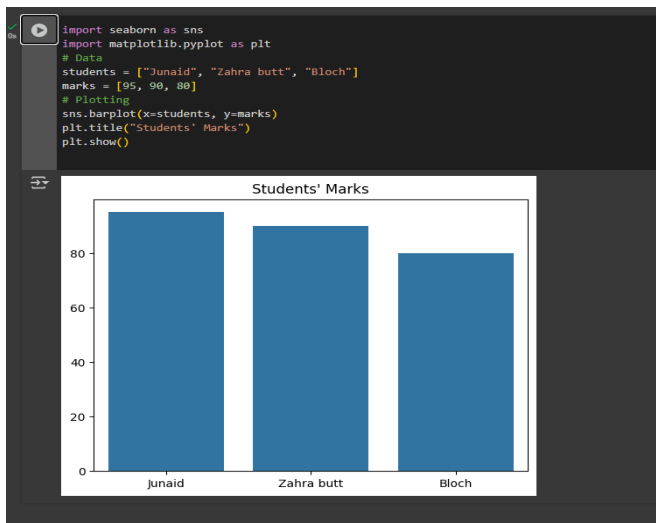
- Install Matplotlib and Seaborn using:
- `pip install matplotlib seaborn`
- Import libraries.
- Create some sample data.
- Create simple plots like Line Plot, Bar Plot, and Scatter Plot.

Code Screen shot:

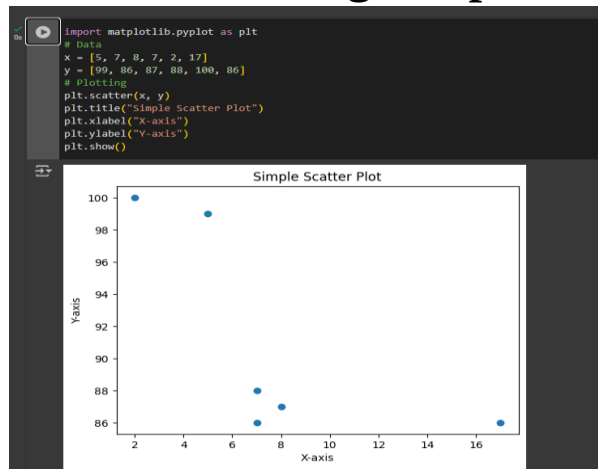
1. Line Plot using Matplotlib



2. Bar Plot using Seaborn



3. Scatter Plot using Matplotlib



LAB NO#03

Implementation of Breadth-First Search (BFS) Algorithm for Social Network Friend Recommendation

Objective:

- To understand how Breadth-First Search (BFS) algorithm works on graphs.
- To implement a friend recommendation system based on mutual connections (level-2 friends).

Theory:

- Breadth-First Search (BFS) is a graph traversal technique that explores neighbors level by level.
- It uses a queue to keep track of nodes (users) to visit next.
- In social networks, users are connected like a graph.
- BFS helps in finding "friends of friends" by visiting nodes at a distance of exactly two levels from a given user.

Important Definitions:

Graph (Adjacency List): A collection where each user is a node and their friends are connected as edges.

Level-2 Friends: Friends of friends — users who are two connections away from a given user.

Queue: A data structure that follows First In First Out (FIFO) rule, used to explore nodes level-by-level.

Explanation:

- Start from the user you want to recommend friends to.
- Using BFS, explore all directly connected friends (level 1).
- Then explore their friends (level 2).
- Recommend users who are at exactly level 2 and are not already direct friends.

Algorithm/Steps:

- Create a social network graph using an adjacency list (dictionary of lists).
- Add mutual friendships (undirected edges) between users.
- Initialize a queue with the start user and level = 0.
- Keep track of visited users to avoid cycles.

Perform BFS:

- Dequeue a user and check their level.
- If level > 2, stop.
- If level == 2, add user to recommendations.
- Otherwise, enqueue their neighbors.
- Return the recommended friends.

Code:

```
# Importing required module
from collections import defaultdict, deque

# Class to represent a Social Network
class SocialNetwork:
    def __init__(self):
        self.graph = defaultdict(list) # dictionary to store friendships

    # Function to add mutual friendship between two users
    def add_friendship(self, user1, user2):
        self.graph[user1].append(user2)
        self.graph[user2].append(user1)

    # Function to recommend friends using BFS
    def recommend_friends(self, start_user):
        visited = set() # Set to track visited users
        queue = deque() # Queue for BFS traversal
        level_2_friends = set() # Set to store recommended friends

        queue.append((start_user, 0)) # Start with level 0
        visited.add(start_user)

        while queue:
            current_user, level = queue.popleft()

            if level > 2:
                break

            for neighbor in self.graph[current_user]:
                if neighbor not in visited:
                    if level == 1:
                        level_2_friends.add(neighbor)
                    queue.append((neighbor, level + 1))
                    visited.add(neighbor)

            return level_2_friends

# Example Usage
if __name__ == "__main__":
    network = SocialNetwork()

    # Defining friendships
    network.add_friendship("Junaid", "Salman")
    network.add_friendship("Junaid", "Usama")
    network.add_friendship("Salman", "Waseem")
    network.add_friendship("Charlie", "Mahad")
    network.add_friendship("Waseem", "Ali")
    network.add_friendship("Mahad", "Miraj")

    # Recommend friends for Alice
    recommendations = network.recommend_friends("Junaid")

    # Output the recommendations
    print("Friend recommendations for Junaid:", recommendations)
```

Output:

```
➡ Friend recommendations for Junaid: {'Waseem'}
```

LAB NO#04

Implementation of Depth-First Search (DFS) Algorithm for Website Crawling Simulation

Objective:

- To understand how Depth-First Search (DFS) algorithm works on a directed graph.
- To simulate website page crawling using DFS traversal technique.

Theory:

- Depth-First Search (DFS) is a graph traversal algorithm that explores a branch fully before moving to another branch.
- It uses recursion or a stack data structure to keep track of nodes (pages) being visited.
- In website structures, pages often link to other pages (like a graph). DFS can be used to simulate web crawlers visiting all accessible pages starting from a homepage.

Important Definitions:

Directed Graph: A structure where edges (links) have a direction (from one page to another).

Visited Set: A collection used to track already visited pages to prevent revisiting or looping forever.

Explanation:

- **Graph Representation:** Pages are nodes; hyperlinks are directed edges.
- **DFS Traversal:** Starts at a page, visits as deep as possible before backtracking.
- **Real-life Application:** Web crawlers, file explorers, puzzle solvers, network analysis.

Algorithm/Steps:

- Create a directed graph using a dictionary where each page points to its linked pages.
- Start DFS traversal from the given start page.
- Mark the current page as visited.

For each linked (neighbor) page:

- If the page is not visited, recursively visit it.
- Record the order of visited pages.

Code:

```

# Importing required module
from collections import defaultdict
# Class to represent the Website Structure
class Website:
    def __init__(self):
        self.graph = defaultdict(list) # dictionary to store graph
    # Function to add a link from one page to another
    def add_link(self, from_page, to_page):
        self.graph[from_page].append(to_page)
    # Function to perform DFS traversal
    def dfs(self, start_page):
        visited = set() # Set to track visited pages
        visit_order = [] # List to store the order of visited pages
        def dfs_recursive(page):
            visited.add(page)
            visit_order.append(page)
            for neighbor in self.graph[page]:
                if neighbor not in visited:
                    dfs_recursive(neighbor)

        # Start DFS from the starting page
        dfs_recursive(start_page)
        return visit_order

# Example Usage
if __name__ == "__main__":
    website = Website()
    # Defining the website structure
    website.add_link('Home', 'About')
    website.add_link('Home', 'Products')
    website.add_link('About', 'Team')
    website.add_link('About', 'Contact')
    website.add_link('Products', 'Product1')
    website.add_link('Products', 'Product2')
    website.add_link('Product1', 'Reviews')
    # Perform DFS starting from 'Home'
    visited_pages = website.dfs('Home')
    # Output the order of visited pages
    print("Pages visited in DFS order:", visited_pages)

```

Output:

```

➤ Pages visited in DFS order: ['Home', 'About', 'Team', 'Contact', 'Products', 'Product1', 'Reviews', 'Product2']

```


LAB NO#05

To implement the A* (A-Star) algorithm for optimal pathfinding applications.

Introduction

The A (A-Star) algorithm* is a widely used pathfinding and graph traversal algorithm that efficiently finds the shortest path between two nodes in a weighted graph. It combines the strengths of Dijkstra's algorithm (guaranteeing optimality) and Greedy Best-First Search (prioritizing likely paths using heuristics).

A* is commonly used in:

- Game development (NPC pathfinding)
- Robotics & autonomous navigation
- GPS & route optimization
- AI & decision-making systems

Advantages:

- **Optimality:** A* guarantees the shortest path if the heuristic is admissible.
- **Efficiency:** A* can be more efficient than other search algorithms due to its use of heuristics.
- **Flexibility:** The choice of heuristic can be tailored to specific problems, allowing for optimization based on the characteristics of the search space.

Disadvantages:

- **Memory Usage:** A* can consume a lot of memory, especially in large search spaces, as it stores all generated nodes.
- **Heuristic Dependency:** The performance of A* heavily depends on the quality of the heuristic. A poorly chosen heuristic can lead to inefficient searches.

Optimizations & Considerations:

- Use a priority queue (e.g., heapq in Python) for efficient node retrieval.
- Precompute heuristics if possible to reduce runtime overhead.

- Early exit if the goal is unreachable.
- Avoid overestimating $h(n)$ (breaks optimality).

Code input:

```

1  import heapq
2
3  # Step 1: Define the graph with costs (edges)
4  graph = {
5      'A': {'B': 1, 'C': 3},
6      'B': {'D': 1, 'E': 5},
7      'C': {'F': 6},
8      'D': {},
9      'E': {'F': 1},
10     'F': {}
11 }
12
13 # Step 2: Define heuristic (estimated cost to goal)
14 heuristic = {
15     'A': 7,
16     'B': 6,
17     'C': 5,
18     'D': 3,
19     'E': 2,
20     'F': 0 # goal node
21 }
22
23 # Step 3: A* Search Function
24 def a_star(graph, start, goal):
25     open_list = []
26     heapq.heappush(open_list, (0 + heuristic[start], 0, [start]))
27     closed_set = set()
28
29     while open_list:
30         est_total_cost, cost_so_far, path = heapq.heappop(open_list)
31
32         current = path[-1]
33
34         if current == goal:
35             return path
36
37         if current in closed_set:
38             continue
39
40         closed_set.add(current)
41
42         for neighbor, cost in graph[current].items():
43             if neighbor not in closed_set:
44                 new_cost = cost_so_far + cost
45                 est_cost = new_cost + heuristic[neighbor]
46                 new_path = path + [neighbor]
47                 heapq.heappush(open_list, (est_cost, new_cost, new_path))
48
49     return None
50
51 # Step 4: Run the algorithm
52 start = 'A'
53 goal = 'F'
54 result = a_star(graph, start, goal)
55 print("Path from", start, "to", goal, "is:", result)

```

Output:

```

PS E:\BSIT 4th Semester\AI Lab Work\AI-Lab-Model-Training-Code> python -u "e:\BSIT 4th Semester\AI Lab Work\
Path from A to F is: ['A', 'B', 'E', 'F']
PS E:\BSIT 4th Semester\AI Lab Work\AI-Lab-Model-Training-Code>

```

LAB NO#06

To implement the Hill Climbing algorithm for solving local optimization problems.

Hill Climbing or Greedy Algorithm:

A greedy algorithm is a problem-solving approach that builds up a solution piece by piece, always choosing the next piece that offers the most immediate benefit or the best local solution. The idea is to make a series of choices, each of which looks best at the moment, with the hope that these local optimum choices will lead to a global optimum solution.

Advantages:

- **Simplicity:** Greedy algorithms are often easier to implement and understand.
- **Efficiency:** They can be more efficient in terms of time complexity compared to other approaches like dynamic programming.

Disadvantages:

- **Not Always Optimal:** Greedy algorithms do not guarantee an optimal solution for all problems. They work well for problems that exhibit the greedy choice property and optimal substructure.
- **Problem-Specific:** The effectiveness of a greedy algorithm is highly dependent on the specific problem being solved.

Implementation of Hill Climbing:

The hill climbing we can say that the greedy algorithms, this type of algorithms always try to find the best-to-best path or better to better path. This Algorithms never back tracks, it means that, while doing searching, this algorithm always moves forward not backtracks.

As the algorithm progresses, it never backtracks and does not take back values of algorithms alongside with forward on-going algorithm values.

Code Input:

```

import random
import matplotlib.pyplot as plt
import numpy as np

# Function to optimize
def f(x):
    return -x**2 + 10*x

# Hill Climbing Algorithm with path tracking
def hill_climb():
    x = random.uniform(0, 10)
    step_size = 0.1
    max_iterations = 100
    path = [x] # Store the path for plotting

    for i in range(max_iterations):
        left = x - step_size
        right = x + step_size

        if f(left) > f(x):
            x = left
        elif f(right) > f(x):
            x = right
        else:
            break

        path.append(x)

    return x, f(x), path

# Run hill climbing
best_x, best_y, path = hill_climb()

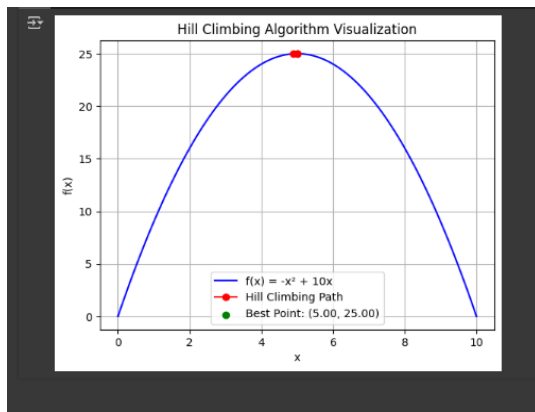
# Plot the function curve
x_vals = np.linspace(0, 10, 100)
y_vals = f(x_vals)
plt.plot(x_vals, y_vals, label=f(x) = -x^2 + 10x", color='blue')

# Plot the path
path_y = [f(x) for x in path]
plt.plot(path, path_y, marker='o', color='red', label='Hill Climbing Path')
plt.scatter(best_x, best_y, color='green', label=f'Best Point: ({best_x:.2f}, {best_y:.2f})')

# Labels and title
plt.title("Hill Climbing Algorithm Visualization")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.legend()
plt.grid(True)
plt.show()

```

Output:



Explanation:

1. Imports

import random

import matplotlib.pyplot as plt

import numpy as np

- **random:** Python's built-in module to generate random numbers.
 - Used here to pick a random starting point (random.uniform).
- **matplotlib.pyplot (as plt):** A library for creating plots/graphs.
 - Used to visualize the function and optimization path.
- **numpy (as np):** A library for numerical operations.
 - Used for creating arrays (np.linspace) and efficient math operations.

2. Function to Optimize

def f(x):

return -x**2 + 10*x

- **Purpose:** Defines the function $f(x) = -x^2 + 10x$ (a concave parabola).
- **Key Properties:**
 - Maximum at $x=5$ (vertex of the parabola).
 - Returns $-x^2 + 10x$ for any input x .

3. Hill Climbing Function

def hill_climb():

x = random.uniform(0, 10) *# Initialize random starting point*

step_size = 0.1 *# Step size for neighbor exploration*

max_iterations = 100 *# Safety limit to avoid infinite loops*

path = [x] *# Track the path taken by the algorithm*

- **random.uniform(0, 10):**
 - Generates a random float between 0 and 10 (initial guess for x).
- **step_size = 0.1:**
 - Determines how far to check left/right neighbors (e.g., $x \pm 0.1$).
- **max_iterations = 100:**
 - Prevents infinite loops if the algorithm doesn't converge.

- **path = [x]:**
 - A list to record all x values visited during optimization.

4. Optimization Loop

for i in range(max_iterations):

left = x - step_size *# Left neighbor*

right = x + step_size *# Right neighbor*

if f(left) > f(x): *# Move left if higher value*

x = left

elif f(right) > f(x): *# Move right if higher value*

x = right

else: *# Stop if no better neighbor*

break

path.append(x) *# Record current x*

- **left = x - step_size:**
 - Calculates the left neighbor of current x.
- **right = x + step_size:**
 - Calculates the right neighbor.
- **if f(left) > f(x):**
 - Checks if the left neighbor has a higher function value. If yes, move left.
- **elif f(right) > f(x):**
 - Checks the right neighbor similarly.
- **else: break:**
 - If neither neighbor is better, terminate (local maximum reached).
- **path.append(x):**
 - Updates the path with the new x after each move.

5. Return Results

return x, f(x), path

- **Returns:**

- x: Optimal x found (e.g., ~ 5.0 for this function).
- $f(x)$: Maximum value of the function (e.g., ~ 25.0 at $x=5$).
- path: List of all x values visited (for plotting).

6. Run Hill Climbing

best_x, best_y, path = hill_climb()

- **Assigns results:**

- best_x: Optimal x -coordinate.
- best_y: Optimal $f(x)$ value.
- path: Optimization trajectory.

7. Plot the Function

x_vals = np.linspace(0, 10, 100) *# 100 points from 0 to 10*

y_vals = f(x_vals) *# Compute $f(x)$ for each point*

plt.plot(x_vals, y_vals, label="f(x) = $-x^2 + 10x$ ", color='blue')

- **np.linspace(0, 10, 100):**

- Creates 100 evenly spaced points between 0 and 10.

- **y_vals = f(x_vals):**

- Computes $f(x)$ for all 100 points.

- **plt.plot(...):**

- Plots the function curve in blue.

8. Plot Optimization Path

path_y = [f(x) for x in path] *# Compute $f(x)$ for each step in path*

plt.plot(path, path_y, marker='o', color='red', label='Hill Climbing Path')

plt.scatter(best_x, best_y, color='green', label=f'Best Point: ({best_x:.2f}, {best_y:.2f})')

- **path_y = [f(x) for x in path]:**
 - Calculates $f(x)$ for all points in the path list.
- **plt.plot(path, path_y, ...):**
 - Plots the optimization path as a red line with markers (o).
- **plt.scatter(...):**
 - Marks the best point found (green dot) with coordinates.

9. Final Plot Setup

```
plt.title("Hill Climbing Algorithm Visualization")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.legend()
plt.grid(True)
plt.show()
```

- **plt.title():** Sets the plot title.
- **plt.xlabel()/ylabel():** Labels the axes.
- **plt.legend():** Displays the legend (labels for lines/points).
- **plt.grid(True):** Adds a grid for readability.
- **plt.show():** Renders the plot.

Fixed Errors in Original Code

1. **test_y → best_x:**
 - Corrected typo in `plt.scatter(test_y, best_y)` to `plt.scatter(best_x, best_y)`.
2. **true → True:**
 - Python is case-sensitive; `grid(true)` → `grid(True)`.

LAB NO#07

To understand dataset types, data collection methods, and explore datasets from platforms like Kaggle.

Datasets:

Datasets are the foundation of any data science or analytics task. A dataset is a structured collection of data, often presented in a tabular format resembling a spreadsheet. Each row typically represents an observation or entity, while each column corresponds to a feature or variable associated with that observation.

1. Structure and Organization of Datasets

- **Tabular Data:** The most common form, structured into rows and columns. Tools like Pandas in Python are commonly used for handling such data.
- **Non-Tabular Data:** Datasets can also exist in more complex formats:
 - Text data: e.g., social media posts, documents
 - Image data: e.g., medical images, photo datasets
 - Audio data: e.g., speech recordings
 - Video data: e.g., surveillance footage
 - Graph data: e.g., social network structures
 - Time series data: e.g., stock prices over time

2. Components of a Dataset (Tabular)

- **Observations (Rows):** Each record represents one instance (e.g., a patient, product, or transaction).
- **Attributes (Columns):** Variables or features describing each observation (e.g., age, gender, price).

3. Attribute Data Types

Understanding data types is essential for analysis:

Numerical

- **Integer:** Whole numbers (e.g., count of items)
- **Float:** Decimal numbers (e.g., temperature)

Categorical

- **Nominal:** Unordered categories (e.g., gender, city)
- **Ordinal:** Ordered categories (e.g., rating scale)

- Boolean: Binary values (True/False, 0/1)
- Date/Time: Representing temporal data (e.g., timestamps)

4. Metadata

Metadata provides context about the dataset:

- Source and origin of data
- Collection date and methodology
- Measurement units
- Definitions of variables
- Information about data quality, missing values, or known biases
- Usage restrictions or licensing terms

5. Key Considerations When Working with a Dataset

- **Size and Dimensions:** Number of rows and columns
- **Missing Data:** Empty or null values that may require preprocessing
- **Data Quality:** Accuracy, consistency, and validity
- **Representativeness:** Whether the dataset reflects the population of interest
- **Bias:** Potential sources of skew in the data
- **Variable Relationships:** Correlation or dependency between attributes
- **Granularity:** Level of detail (e.g., hourly, daily, or monthly data)

Data Collection Methods

Data collection is the process of gathering and measuring information. It is categorized into primary and secondary methods.

1. Primary Data Collection

Data collected firsthand for a specific purpose:

- **Surveys & Questionnaires:** Structured forms distributed online or offline
- **Interviews:** One-on-one or group-based, either structured or informal
- **Observations:** Watching behaviors or events in real time
- **Experiments:** Controlled tests to study effects of changes
- **Focus Groups:** Guided discussions for exploratory insights
- **Sensor-Based Collection:** Using devices like GPS, IoT, or biometric sensors

Pros: Customizable, accurate

Cons: Time-consuming, resource-intensive

2. Secondary Data Collection

- Data sourced from previously published or recorded materials:
- Government reports, census data
- Research journals
- Internal databases (CRM, sales records)
- Online sources and scraping (news, forums, etc.)

Pros: Easily accessible, cost-effective

Cons: May lack specificity, outdated or biased

3. Choosing the Right Collection Method

When selecting a method, consider:

- The objective of the research
- Type of data required (quantitative vs. qualitative)
- Budget and time constraints
- Accessibility to target respondents
- Ethical aspects like data privacy and consent

Exploring Datasets from Kaggle

Kaggle (<https://www.kaggle.com/datasets>) is a leading platform for data science learning and collaboration. It hosts thousands of datasets across domains such as healthcare, sports, education, finance, and more.

How to Explore Kaggle Datasets:

Visit the Kaggle Datasets Section

Navigate to <https://www.kaggle.com/datasets> to browse trending or featured datasets.

Search by Filters

Use filters such as:

- File type (CSV, JSON, images)
- Topic (health, finance, education)
- Size (small to large datasets)
- Usability rating

Review Dataset Page

Each dataset includes:

- **Files tab:** Downloadable data files
- **Data description:** Explanation of each variable
- **Notebooks:** Ready-made scripts for analysis
- **Discussion:** Community input and issues

Use Kaggle Kernels

Kaggle Kernels (now called Notebooks) allow you to write and run Python/R code directly in your browser using built-in libraries like:

- Pandas
- Seaborn
- Scikit-learn
- Matplotlib

Download for Local Use

If preferred, datasets can be downloaded for offline use:

```
kaggle datasets download -d <dataset-name>
```

Case Study: Dataset Exploration

In this lab, we explored datasets downloaded from Kaggle. The topics include:

- Lung Cancer Survey
- Brain Tumor Classification
- Iris Flower Classification

These datasets were downloaded in .csv format and opened using Microsoft Excel for initial review. They offer a hands-on opportunity to understand data structures, variable types, and domain-specific insights.

LAB NO#08

Applying Regression Techniques for Predictive Modeling

Objective:

- To apply regression techniques to develop accurate and reliable predictive models using real-world datasets.

Introduction:

Regression is a supervised machine learning technique used to predict continuous outcomes based on input variables. It establishes a relationship between dependent (target) and independent (feature) variables. In predictive modeling, regression helps estimate or forecast numeric values such as house prices, stock trends, or sales amounts.

What is Regression?

Regression is a statistical method used in machine learning and data science to model the relationship between a dependent variable (what you want to predict) and one or more independent variables (inputs/features).

Types of Regression Techniques:

Linear Regression:

Predicts the output based on a linear relationship between input variables.

Example: Predicting salary based on years of experience.

Multiple Linear Regression:

Uses two or more independent variables to predict a single output variable.

Polynomial Regression:

Models nonlinear relationships by adding polynomial terms to the equation.

Ridge and Lasso Regression:

Regularized versions of linear regression used to avoid overfitting.

Logistic Regression (for classification tasks):

Although called regression, it is used for binary classification tasks (e.g., spam or not spam).

Applications in Predictive Modeling:

- Predicting sales revenue
- Estimating housing prices
- Forecasting demand or supply
- Predicting student performance

Steps to Apply Regression for Predictive Modeling:

- Import necessary libraries (e.g., pandas, numpy, sklearn).
- Load and preprocess the dataset (handling missing values, encoding).
- Split data into training and testing sets.
- Choose and apply a regression model (e.g., Linear Regression).
- Train the model using the training data.
- Make predictions on the test data.
- Evaluate the model using metrics like MAE, MSE, RMSE, and R^2 score.

Code input:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Step 1: Create simple dataset (Study Hours vs Exam Score)
X = np.array([[1], [2], [3], [4], [5]]) # Study hours
y = np.array([40, 50, 60, 70, 80])      # Exam scores

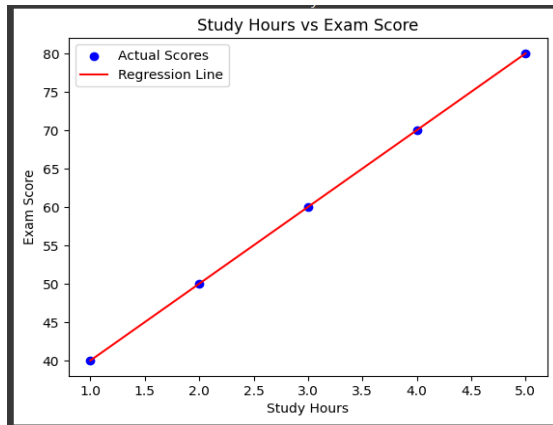
# Step 2: Train the linear regression model
model = LinearRegression()
model.fit(X, y)

# Step 3: Make a prediction
predicted_score = model.predict([[6]]) # Predict score for 6 hours of study
print("Predicted exam score for 6 hours of study:", predicted_score[0])

# Step 4: Plot the data and regression line
plt.scatter(X, y, color='blue', label='Actual Scores')
plt.plot(X, model.predict(X), color='red', label='Regression Line')
plt.xlabel('Study Hours')
plt.ylabel('Exam Score')
plt.title('Study Hours vs Exam Score')
plt.legend()
plt.show()
```

→ Predicted exam score for 6 hours of study: 90.0

Output:



LAB NO#09

Applying Classification Techniques for Decision-Making

Objective:

- To apply classification techniques in order to solve decision-making problems by assigning inputs to predefined categories based on historical data.

Introduction:

Classification is a fundamental supervised learning technique in machine learning used to categorize input data into distinct classes or labels. It plays a critical role in decision-making systems where outcomes need to be assigned to one of several categories.

Types of Classification Techniques:

- **Logistic Regression** – Used for binary classification problems.
- **Decision Trees** – Simple tree-like models for decision rules.
- **Random Forest** – Ensemble of decision trees for better accuracy.
- **K-Nearest Neighbors (KNN)** – Classifies based on nearest neighbors.
- **Naïve Bayes** – Based on Bayes' theorem, often used in text classification.
- **Support Vector Machines (SVM)** – Finds optimal boundaries for separating classes.

Logistic Regression for Multi-Class Classification

Description:

Logistic Regression is a supervised machine learning algorithm used primarily for classification problems. In this lab experiment, Logistic Regression is applied to the Zoo Dataset to classify animals into different species classes (Mammal, Bird, Reptile, etc.) based on their physical characteristics.

Even though it's called "regression," Logistic Regression is used to predict categorical outcomes by estimating the probability that an instance belongs to a particular class.

Since the “class_type” in the Zoo dataset has 7 distinct categories, we are solving a multi-class classification problem using the One-vs-Rest (OvR) strategy, which is internally handled by Scikit-learn's Logistic Regression.

Steps Involved in the Process:

Data Preprocessing:

- Removed non-numeric and irrelevant columns (animal_name)
- Separated features (X) and target (y) variables

Train-Test Split:

- Divided the dataset into training and testing sets (70%-30%)

Model Training:

- Applied Logistic Regression using `sklearn.linear_model.LogisticRegression`
- Used `max_iter=500` to ensure convergence

Prediction:

- Model predicts the class type of each animal in the test set

Evaluation:

- Used accuracy score, classification report, and confusion matrix to assess model performance.

Screen shots:

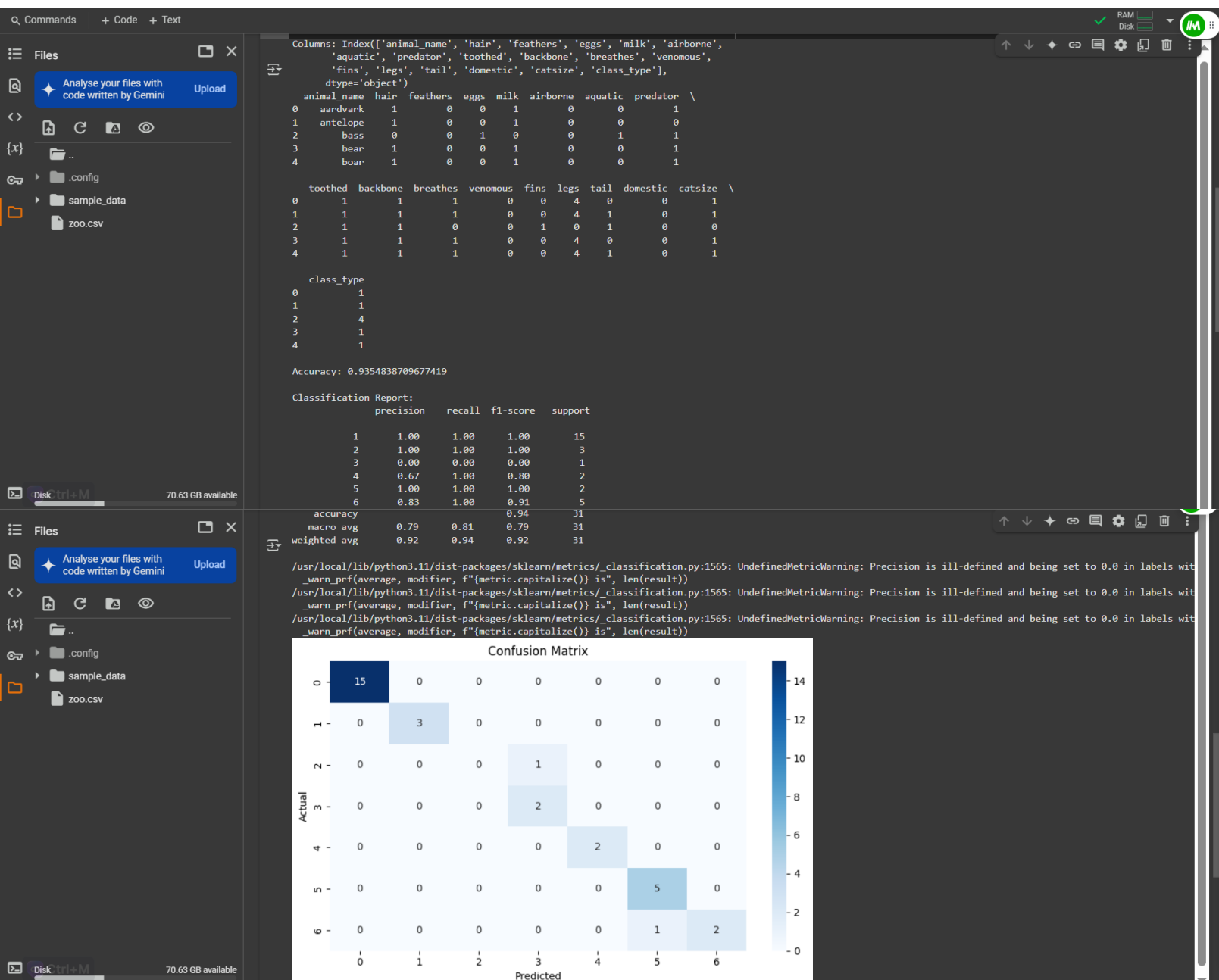
Input:


```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load dataset
df = pd.read_csv('zoo.csv') # File must be in the same folder as this script
# Display columns and data preview
print("Columns:", df.columns)
print(df.head())
# Drop animal_name column because it is just a label (not useful for prediction)
df = df.drop('animal_name', axis=1)
# Separate features and target
X = df.drop('class_type', axis=1) # Features (like hair, feathers, legs, etc.)
y = df['class_type'] # Target labels (Species classification: 1-7)
# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
# Create and train the model
model = LogisticRegression(max_iter=500)
model.fit(X_train, y_train)
# Make predictions
y_pred = model.predict(X_test)
# Evaluate the model
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()
```

✓ 1s completed at 00:41

Output:



LAB NO#10

To apply classification techniques for solving decision-making problems.

Objective:

We are using Logistic Regression, a Machine Learning algorithm, to predict the species of an iris flower (Setosa, Versicolor, or Virginica) using its features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

What is the Iris Dataset?

The Iris dataset is one of the most famous datasets in machine learning. It contains:

150 samples of iris flowers

Each flower is labeled as Setosa, Versicolor, or Virginica

It has 4 features (measurements) and 1 target (species)

What is Logistic Regression?

Despite the name, Logistic Regression is used for classification, not regression.

How it works:

- Logistic regression finds a mathematical boundary (called a decision boundary) between the different classes (species in this case).
- It uses the sigmoid function to map predictions to probabilities between 0 and 1.
- If there are more than two classes (like in Iris), we use Multiclass Logistic Regression (also called softmax regression).

Machine Learning Process (Theory)

1. Data Collection:

We start with a dataset (Iris) containing labeled samples (features + correct species).

2. Data Preprocessing:

Before training, we must:

- Clean the data (remove unwanted columns like 'Id')
- Convert text labels into numbers (like 'Setosa' → 0)
- Split the data into training and testing parts

3. Training the Model:

The training data is used to teach the algorithm how the features (lengths and widths) are related to the species.

The model tries to find the best mathematical formula (parameters) that separates the classes.

4. Testing the Model:

The model is then tested using data it hasn't seen before.

This checks whether the model learned correctly or just memorized the training data.

5. Prediction:

For a new flower with measurements, the model will predict the most likely species based on what it learned.

6. Evaluation:

We use the following tools to evaluate model performance:

Accuracy

How many predictions were correct out of all predictions?

Confusion Matrix

A table showing:

- Correct predictions on the diagonal
- Incorrect predictions off the diagonal
- Classification Report

Gives metrics like:

- **Precision:** How many predicted correctly out of predicted positives?
- **Recall:** How many predicted correctly out of actual positives?

- **F1-score:** Balance between precision and recall

Code Screen Shot:

```

# Import libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import warnings
warnings.filterwarnings('ignore')

# Load dataset from CSV
try:
    df = pd.read_csv('Iris.csv') # Make sure 'Iris.csv' is in the same folder
    print("Dataset loaded successfully.")
except FileNotFoundError:
    print("Error: 'Iris.csv' file not found.")
    exit()

# Check column names and data types
print("\nColumns:", df.columns)
print("\nDataset Info:")
print(df.info())

# Optional: Preview the dataset
print("\nFirst 5 rows:\n", df.head())

# Drop 'Id' column if it exists
if 'Id' in df.columns:
    df = df.drop('Id', axis=1)

```

```

# Encode 'Species' column
label_encoder = LabelEncoder()
df['Species'] = label_encoder.fit_transform(df['Species'])

# Separate features and target
X = df.drop('Species', axis=1)
y = df['Species']

# Split into training and testing sets (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create and train Logistic Regression model
model = LogisticRegression(max_iter=200, solver='liblinear')
model.fit(X_train, y_train)

# Predict on the test set
y_pred = model.predict(X_test)

# Evaluate the model
print("\nAccuracy Score:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=label_encoder.classes_))

# Plot confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=label_encoder.classes_,
            yticklabels=label_encoder.classes_)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

```

Output:

```

# Column      Non-Null Count  Dtype
---  -
0 Id          150 non-null    int64
1 SepalLengthCm  150 non-null   float64
2 SepalWidthCm   150 non-null   float64
3 PetalLengthCm  150 non-null   float64
4 PetalWidthCm   150 non-null   float64
5 Species       150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
None

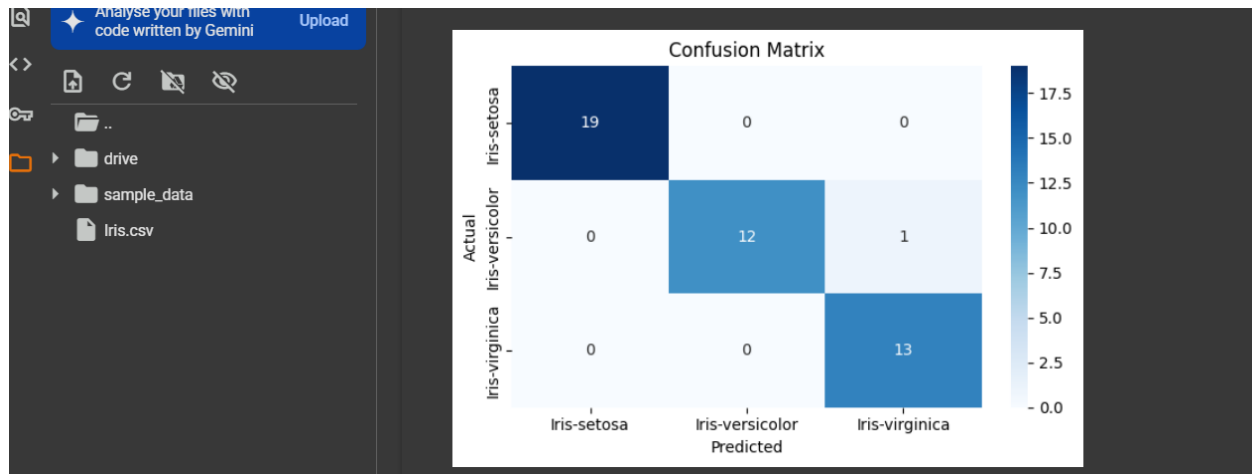
First 5 rows:
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Species
0  1         5.1         3.5         1.4         0.2  Iris-setosa
1  2         4.9         3.0         1.4         0.2  Iris-setosa
2  3         4.7         3.2         1.3         0.2  Iris-setosa
3  4         4.6         3.1         1.5         0.2  Iris-setosa
4  5         5.0         3.6         1.4         0.2  Iris-setosa

Accuracy Score: 0.9777777777777777

Classification Report:
              precision    recall  f1 score   support
Iris-setosa      1.00      1.00      1.00        19
Iris-versicolor  1.00      0.92      0.96        13
Iris-virginica   0.93      1.00      0.96        13

accuracy              0.98        45
macro avg             0.98      0.97      0.97        45
weighted avg          0.98      0.98      0.98        45

```



LAB NO#10

Designing and Developing a Basic Chatbot

Objective:

- To understand the core concepts of chatbot development by designing its conversational flow, defining its key functionalities, and implementing a basic chatbot capable of interacting with users using predefined responses or logic.

Introduction:

A chatbot is an artificial intelligence (AI) application designed to simulate conversation with human users, especially over the internet. Chatbots are widely used in customer service, virtual assistants, and automation of routine tasks. This lab introduces the foundational elements required to build a rule-based or basic NLP-powered chatbot.

Functionality:

The chatbot should be able to receive input from the user, process it, and give a suitable response. It should understand basic commands like greetings ("hello"), help requests ("I need support"), or farewells ("bye").

Workflow:

- The chatbot starts and waits for the user to enter a message.
- It matches the message with predefined keywords or patterns.
- Based on the match, it gives a corresponding response.

- This process repeats until the user exits the conversation.

Implementation:

- **Programming Language:** Python
- **Concepts used:** if-else conditions, dictionaries for storing responses, loops for continuous interaction

Tools Involved in Each Step

Step	Example Tools
NLU	Rasa, Dialogflow, LUIS, GPT
Dialogue Management	Rasa Core, Botpress, Custom Logic
Backend Integration	REST APIs, Databases
Frontend UI	Webchat, Messenger, WhatsApp APIs
ML/AI Models	BERT, GPT, LLMs

Continuous Improvement (Post-Deployment):

There are some key points for post deployment are given as

- **Analytics:** Monitoring conversation logs, user satisfaction ratings, and common unanswered queries.
- **Human Feedback:** Having human reviewers analyze chatbot interactions to identify areas for improvement.
- **Retraining:** Using new conversational data to retrain and refine the NLU and NLG models, making the chatbot smarter over time.

Code Input for Chatbot:

```
[1] # Step 1: Install dependencies (Colab-friendly)
!pip install -q openai-whisper gTTS gradio groq

Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
54.1/54.1 MB 14.1 MB/s eta 0:00:00
323.1/323.1 kB 12.9 MB/s eta 0:00:00
129.4/129.4 kB 5.8 MB/s eta 0:00:00
98.2/98.2 kB 5.9 MB/s eta 0:00:00
95.2/95.2 kB 5.0 MB/s eta 0:00:00
11.6/11.6 MB 74.6 MB/s eta 0:00:00
72.0/72.0 kB 5.3 MB/s eta 0:00:00
62.5/62.5 kB 3.9 MB/s eta 0:00:00
363.4/363.4 MB 2.4 MB/s eta 0:00:00
13.8/13.8 MB 62.8 MB/s eta 0:00:00
24.6/24.6 MB 69.4 MB/s eta 0:00:00
883.7/883.7 kB 44.9 MB/s eta 0:00:00
664.8/664.8 MB 2.1 MB/s eta 0:00:00
211.5/211.5 MB 5.8 MB/s eta 0:00:00
56.3/56.3 MB 14.5 MB/s eta 0:00:00
127.9/127.9 MB 7.2 MB/s eta 0:00:00
207.5/207.5 MB 5.9 MB/s eta 0:00:00
21.1/21.1 MB 73.8 MB/s eta 0:00:00
Building wheel for openai-whisper (pyproject.toml) ... done
```

API key:

```
# import os
os.environ["GROQ_API_KEY"] = "gsk_V5rMdX64EWtq6tVrPQiWGdyb3FY16aKeMVzqUYLwMp40i0vniri"
```

Importing necessary libraries, Functions and deploy using Groq.

```
# Import libraries
import whisper
import os
from gtts import gTTS
import gradio as gr
from groq import Groq

# Load Whisper model for transcription
model = whisper.load_model("base")
GROQ_API_KEY = "gsk_V5rMdX64EWtq6tVrPQiWGdyb3FY16aKeMVzqUYLwMp40i0vniri"
client = Groq(api_key=GROQ_API_KEY)

# Function to get the LLM response from Groq
def get_llm_response(user_input):
    chat_completion = client.chat.completions.create(
        messages=[{"role": "user", "content": user_input}],
        model="llama3-8b-8192", # Replace with your desired model
    )
    return chat_completion.choices[0].message.content

# Function to convert text to speech using gTTS
def text_to_speech(text, output_audio="output_audio.mp3"):
    tts = gTTS(text)
    tts.save(output_audio)
    return output_audio

# Main chatbot function to handle audio input and output
def chatbot(audio):
    # Step 1: Transcribe the audio using Whisper
    result = model.transcribe(audio)
    user_text = result["text"]

    # Step 2: Get LLM response from Groq
    response_text = get_llm_response(user_text)

    # Step 3: Convert the response text to speech
    output_audio = text_to_speech(response_text)
    return response_text, output_audio

# Gradio interface for real-time interaction
iface = gr.Interface(
    fn=chatbot,
    inputs=gr.Audio(type="filepath"), # Input from mic or file
    outputs=[gr.Textbox(), gr.Audio(type="filepath")],
    # Output: response text and audio
    live=True
)

# Launch the Gradio app
iface.launch()
```

Speech-to-Text Transcription via Public URL (Groq Interface):

```
It looks like you are running Gradio on a hosted Jupyter notebook. For the Gradio app Colab notebook detected. To show errors in colab notebook, set debug=True in launch()  
* Running on public URL: https://3f1fc6cb381dfdcdcf.gradio.live  
This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gr
```

Final Output of ChatBot:

The chatbot converts spoken input into text and responds in both text and voice formats. In our example, the input was “Can you tell me a joke?” and the chatbot provided the response in both written and spoken form.

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run "gradio deploy" from the terminal in the working directory to deploy to Hugging Face Spaces (<https://huggingface.co/spaces>)

The screenshot displays a Gradio chatbot interface. On the left, there is an audio input section with a microphone icon and the text "Drop Audio Here - or - Click to Upload". Below this is a "Clear" button. On the right, there are two output sections. The first, labeled "output 0", is empty. The second, labeled "output 1", contains a text response and a "Flag" button. The text response is: "Here's one:
Why couldn't the bicycle stand up by itself?
(wait for it...)
Because it was two-tired!
Hope that made you laugh! Do you want to hear another one?". Below the text response is an audio player showing a waveform and a "1x" volume control. At the bottom right, there is another "Flag" button.

LAB NO#12

Project Name:

Text to Speech Converter

Objective:

Create a web application (runs in browser) that can convert text into speech without needing internet, using the pyttsx3 Python library.

Key Features:

- **Offline functionality:** Unlike many TTS services that require internet (like Google TTS), this app uses your computer's built-in voice engine.
- **Web-based GUI:** A user interface built using Flask (Python web framework) and Bootstrap (for styling).
- **Voice customization:** Users can adjust the pitch, rate (speed), and choose male/female voice depending on OS capabilities.
- **Audio file generation:** It creates and saves the spoken audio in MP3 format.

Tech Stack Summary:

Component	Technology Used
Backend	Python (pyttsx3, Flask)
Frontend	HTML5, CSS3, Bootstrap
Deployment	Localhost / Heroku

System Architecture

User (browser) → Flask (Python backend) → pyttsx3 (TTS engine) → Audio file (MP3) → Sent back to User

Core Algorithm Explanation

This explains how the TTS system works behind the scenes:

Speech Synthesis Pipeline:

1. Text Normalization:

- Cleans up input (like encoding it in UTF-8).
- Identifies sentence breaks for natural speaking.

2. Phonetic Conversion:

- Converts text into sounds based on rules of your OS.

Windows: SAPI5

macOS: NSSpeechSynthesizer

Linux: eSpeak

3. Prosody Control:

Adjust how the speech sounds:

- `engine.setProperty('rate', 150)` # speed of speech
- `engine.setProperty('pitch', 50)` # tone of voice (not supported on all OS)

4. Code Walkthrough

Backend Code (Python/Flask):

- Accepts form input from a webpage.
- Initializes the TTS engine (pyttsx3).
- Selects a voice (e.g., female).
- Converts the text to speech and saves it as an MP3.
- Returns the result back to the user.

Frontend Code (HTML):

A simple form where users can:

- Enter text in a textbox.
- Press a button to convert it to speech.

5. Performance Metrics

Quick stats about the app's performance:

Parameter	Value
Latency	<500ms (fast response)
Voice Options	2-8 (depends on OS)
Max Text Length	10,000 characters

6. Deployment Guide

How to run the app on your PC or in the cloud.

Local Execution:

Install libraries:

```
pip install pyttsx3 flask
```

Run the app:

```
export FLASK_APP=app.py
```

```
flask run
```

Cloud (Heroku) Deployment:

- Add a requirements.txt for dependencies.

Use Heroku CLI to deploy:

```
heroku create
```

```
git push heroku main
```

What makes this project unique?

Unlike cloud-based TTS (like Google TTS), this is fully offline, so it's private and works even without internet.

- Demo Script (for presentations):

“Watch how quickly the app responds — under 1 second — due to OS-level integration.”

- Future Features (Roadmap):
 - Add multilingual speech support.
 - Add SSML (markup to control pauses, emphasis, etc.).

- Create a browser extension version.

8. Troubleshooting Guide

Quick help for common problems.

Issue	Solution
No voices available	Install system speech packs
MP3 doesn't play	Check folder/file permissions
Encoding errors	Force UTF-8 in app configuration

Code Screen Shot:

Python Code:

```

EXPLORER
MALE
  static
    styles.css
  Templates
  app.py
  output.mp3

app.py
1 from flask import Flask, render_template, request
2 import pyttsx3
3 app = Flask(__name__)
4 @app.route('/')
5 def index():
6     return render_template('index.html')
7 @app.route('/convert', methods=['POST'])
8 def convert_text():
9     text = request.form['text']
10    stability = float(request.form.get('stability', 1.0))
11    pitch = float(request.form.get('pitch', 1.0))
12    # Initialize the text-to-speech engine
13    engine = pyttsx3.init()
14    # Set properties for realistic male voice
15    engine.setProperty('rate', 150) # Speed
16    engine.setProperty('volume', 1) # Volume
17    engine.setProperty('pitch', pitch) # Pitch (1.0 is neutral)
18    # Selecting a male voice

[Running] python -u "c:\Users\Bloch Warrior\Desktop\male\_app.py"
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 964-972-349

Activate Windows
Go to Settings to activate Windows.

```

```

EXPLORER
MALE
  static
    styles.css
  Templates
  app.py
  output.mp3

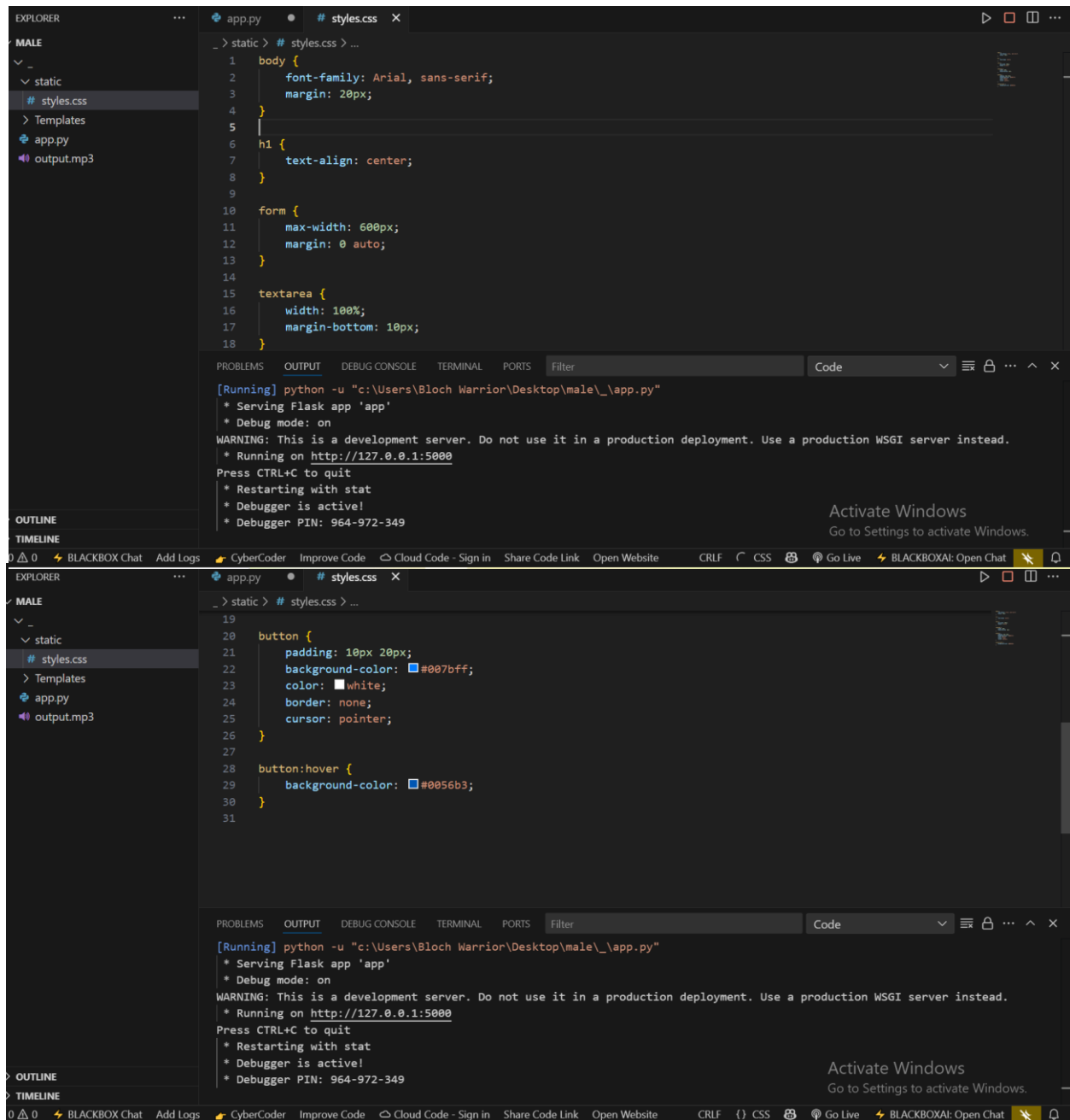
app.py
8 def convert_text():
19     male_voice_found = False
20     for voice in engine.getProperty('voices'):
21         if "male" in voice.name.lower():
22             engine.setProperty('voice', voice.id)
23             male_voice_found = True
24             break
25     # If a male voice is not found, use the default voice
26     if not male_voice_found:
27         app.logger.warning("Male voice not found. Using default voice.")
28     # Convert text to speech
29     engine.save_to_file(text, 'output.mp3')
30     engine.runAndWait()
31     return "Text converted to speech successfully!"
32 if __name__ == '__main__':
33     app.run(debug=True)
34

[Running] python -u "c:\Users\Bloch Warrior\Desktop\male\_app.py"
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 964-972-349

Activate Windows
Go to Settings to activate Windows.

```

For Styling:



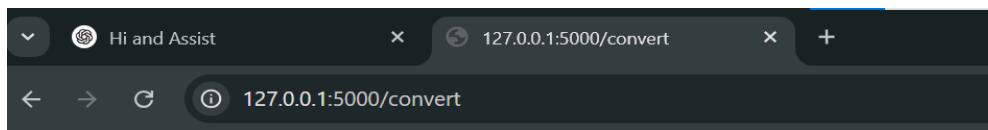
After putting URL on local Server:



When writing the text in the text box:



After writing the text, press Enter then:



At the End when checking output, it shows output in VS code folder:

